

Verifier-Assisted versus One-Shot LLM Intent→Configuration: A Paired NetOpsTraceBench Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed NetOpsTraceBench, a paired comparison of a one-shot LLM intent→configuration pipeline and a verifier-assisted generate-validate-repair pipeline on 1,200 intent→config episodes. Using gpt-5-mini, a deterministic NetOps validator, and Mininet-style emulation, we applied a preregistered binary episode success rule (validator accepts final config AND emulator shows no availability degradation above tolerance AND no automatic rollback). Of 1,200 planned paired tasks we completed 1,199 pairs. Baseline success = 1,196/1,199 (0.9974979149); guarded success = 1,199/1,199 (1.0000000000). The paired-success-rate difference (guarded - baseline) = 0.0025020851 (95% bootstrap percentile CI [0.0000, 0.0058381985], bootstrap_seed=1729, resamples=10,000); exact McNemar two-sided $p = 0.25$. The preregistered power target sought a 5 percentage-point minimum detectable effect; given the observed near-ceiling baseline, the study lacked power to detect much smaller absolute gains. Representative validator traces and provider audit logs show repair was invoked only three times. Under these controlled emulation and task-corpus conditions we find no statistically significant advantage for the verifier-assisted pipeline; we report artifacts, analytic deviation (Wilcoxon → exact McNemar for binary outcomes), and detailed execution accounting to support reproducible interpretation.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent expressed in natural language into executable device configurations for network operations (NetOps). Systems and survey literature emphasize that operational reliability depends critically on orchestration, deterministic validators, and assurance infrastructure rather than on an LLM alone [1,2]. Generate-validate-repair (verifier-assisted) architectures are a canonical mitigation strategy: an LLM proposes candidate outputs, an automated verifier checks constraints and safety, and an automated repair loop iterates until the candidate satisfies verifier rules [3–5]. Despite conceptual support, publicly archived, preregistered, paired evidence measuring whether verifier assistance yields an operationally meaningful improvement (rollback-aware binary success) relative to one-shot generation is sparse.

We address this gap with NetOpsTraceBench: a preregistered, paired, emulator-grounded study comparing (A) baseline one-shot NL→config generation and (B) a guarded generate-validate-repair pipeline using the same shared initial candidate. The study asked: Can a reproducible, workflow-centered benchmark of paired intent→config episodes produce an objective paired binary success metric that reliably distinguishes verifier-assisted pipelines from one-shot generation across two

simulated topologies? Our contributions are (1) a preregistered experimental design and full execution artifacts (study ID rX4f7-1); (2) a paired analysis on 1,199 completed pairs using gpt-5-mini and a deterministic validator; (3) transparent reporting of an analytic deviation (final confirmatory test used an exact McNemar test appropriate for paired binary outcomes) and execution accounting that explains why the guarded pipeline rarely required repair.

II. RELATED WORK

Recent surveys and system papers position LLMs within AIOps/NetOps pipelines and emphasize the importance of verifiers, provenance, and end-to-end workflow evaluation [1,6]. Work on verifier-guided NL→formal translations and intent→topology compilation has shown the feasibility of generate-validate-repair approaches in domains such as security topology compilation and CTL/specification generation [3,7]. Empirical and conceptual studies document LLM failure modes relevant to NetOps—hallucination, incorrect semantics/units, overconfidence, provenance loss, and prompt injection—which motivate guarded architectures and runtime auditing [8–10]. Several recent prototypes demonstrate LLM-assisted configuration generation and misconfiguration detection but typically evaluate on constructed or small held-out sets [4,5,11]. That literature motivates workflow-centered evaluation metrics (trace quality, rollback measurement, canaries) rather than one-shot QA metrics; NetOpsTraceBench operationalizes a paired, rollback-aware binary success metric in a preregistered framework and archives the run artifacts for reproducibility.

III. METHODOLOGY

Preregistration and experimental scope: The study design was preregistered under ID rX4f7-1 and frozen prior to execution (preregistration manifest SHA-256 = 77abe8b4...). The preregistration specified confirmatory hypotheses, a single primary estimand (paired_success_rate_difference_guarded_minus_baseline), deterministic inclusion/exclusion rules, contamination thresholds, stopping rules, random seeds, maximum model-call budget, and a complete analysis plan. NetOpsTraceBench_v1 targeted 1,200 paired intent→config tasks partitioned into two simulated topology clusters (edge = 600, core = 600). Task examples were public synthetic or consented; every task

carried deterministic provenance metadata. The preregistered power plan assumed a baseline success near 0.70 and set a minimum detectable effect (MDE) of 5 percentage points (absolute) at two-sided $\alpha=0.05$ and power 0.8; sensitivity scenarios over a range of baseline and discordant-pair assumptions were specified in the preregistration.

Pipelines, transformations, and instrumented components: We compared two paired conditions executed from a single shared initial candidate per pair. Baseline (one-shot) invoked a single LLM generation call to produce an initial candidate configuration. Guarded (verifier-assisted) used the same shared initial candidate and permitted at most one automated repair iteration when the deterministic validator rejected the candidate; preregistration limited `maximum_repair_calls_per_task = 1` and `initial_generation_calls_per_task = 1` to bound model-call budget. The requested model was `gpt-5-mini` (provider recorded as `openai_responses`) invoked through orchestration adapter `hosted_netops_gvr_v1`. `Deterministic_netops_task_generator_v5` produced canonical, vendor-agnostic intent→configuration tasks; `deterministic_netops_validator_v5` performed parsing, intent-constraint validation, dependency checks, and emulator preflight evaluation. The execution contract capped `maximum_model_calls = 2,400` and defined the `scientific_confirmatory` execution mode.

Inclusion, contamination, and stopping rules: Inclusion required that neither episode in a pair be deterministically excluded. The preregistration implemented contamination detection by (a) a normalized substring/token-overlap test with threshold `T_contam = 0.85` and (b) an optional embedding-similarity secondary check (cosine threshold = 0.92) for flagged items. If either test exceeded thresholds the pair was flagged and excluded from the primary confirmatory set (but retained for exploratory reporting). Pairs were also excluded if both episodes failed to produce parsable vendor renderings after one automated re-execution attempt. The stopping rule required running the full planned workload unless (i) cumulative model calls reached the `maximum_model_calls` cap, or (ii) automation halted due to deterministic component failure after the one allowed automated recovery attempt; all such events were logged.

Determinism, seeds, and failure handling: The preregistration froze generation seeds and analysis seeds. Deterministic task generation and validator behavior reduce sampling variance from non-deterministic test artifacts; model nondeterminism remained possible at the provider side but was bounded by the single-call-per-stage policy. Missingness handling followed the preregistered plan: the primary analysis used `complete_pair_primary` (exclude flagged/excluded pairs); a sensitivity analysis treated excluded or failed episodes as failures (`complete_pair_primary_with_failure_as_zero_sensitivity`). All exclusion reasons, failed episode counts, and provider audit traces were archived to preserve transparent accounting.

Success metric, confirmatory test, and bootstrap CI: The

preregistered binary episode success rule required three conjunctive outcomes: (1) final configuration accepted by the deterministic validator (parsing and intent constraints satisfied), (2) emulator execution shows availability degradation no greater than the preregistered tolerance, and (3) no automatic rollback event occurred during simulated execution. Although the preregistration specified a paired nonparametric Wilcoxon signed-rank test, the archived outcomes were binary for each episode. Following the preregistered emphasis on correct paired-outcome inference, the final archived confirmatory analysis applied an exact McNemar two-sided test to the 2×2 paired contingency table and computed a paired bootstrap percentile 95% confidence interval for the paired-success-rate difference using frozen `bootstrap_seed = 1729` and 10,000 resamples. This analytic deviation (Wilcoxon → exact McNemar) is recorded in the analysis artifacts and in the Disclosure Statement.

Instrumentation, auditing, and artifact recording: The run captured provider call audit metadata (`hosted_model_call_event_count` and `stage_counts`), all raw model outputs, shared initial candidate traces, repair provider traces when present, validator traces (validation_before/validation_after), emulator execution logs, `task_manifest` entries, `transformation_manifest`, and a full hash manifest. Key configured limits recorded in `model_configuration.json` included `max_output_tokens = 1500` and `requested_model = gpt-5-mini`. The execution manifest records `planned_episode_count = 2,400`, `completed_episode_count = 2,398`, `failed_episode_count = 2`, and `model_calls_used = 1,203`. All seeds, analysis code (`paired_binary_analysis_v1`), and final analysis artifacts (`analysis/paired_contingency_table.csv`, `analysis/results.json`, `analysis/paired_difference_figure.svg`) are archived to enable exact reproduction.

Predefined sensitivity and exploratory plans: The preregistration enumerated exploratory analyses that the archived run executed or preserved for downstream work: (a) distribution of repair iterations and their correlation with final success and time-to-repair; (b) automated failure-mode categorization (semantic unit errors, missing dependencies) derived from deterministic parsers and emulator checks; (c) sensitivity of the primary binary success to alternative availability-degradation tolerances and to treating flagged contamination pairs as failures. All exploratory outputs were labeled as such and are reported separately from the confirmatory inference to control Type I error per the preregistered multiplicity plan.

IV. RESULTS

Execution and pair accounting: The run completed 2,398 of 2,400 planned episodes and produced 1,199 complete paired observations (`completed_pairs = 1,199`; `failed_episode_count = 2`). The analysis used `complete_pair_count = 1,199`. Provider audit and orchestration logs record `hosted_model_call_event_count = 1,203` (`completed_hosted_model_call_event_count = 1,202`; `failed_hosted_model_call_event_count = 1`) and `stage_counts`

indicating `shared_initial_generation = 1,200` and `repair = 3`. These counts show that a single shared initial generation was produced per task and that the guarded pipeline invoked repair calls only three times across the entire corpus.

Primary confirmatory outcomes (paired contingency and rates): The paired 2×2 contingency table (analysis/paired_contingency_table.csv) is: $n_{11} = 1,196$ (both methods succeeded), $n_{10} = 3$ (guarded succeeded while baseline failed), $n_{01} = 0$ (baseline succeeded while guarded failed), $n_{00} = 0$ (both failed). From these counts: `baseline_success_count = 1,196` (`baseline_success_rate = 1,196/1,199 = 0.9974979149291076`) and `guarded_success_count = 1,199` (`guarded_success_rate = 1,199/1,199 = 1.0`). The observed paired-success-rate difference (guarded - baseline) = 0.002502085070892446.

Exact paired inference and bootstrap CI: Because confirmatory outcomes are binary and the observed data are concentrated in concordant cells, we used an exact McNemar two-sided test on the discordant pair counts ($n_{10} = 3$, $n_{01} = 0$). The archived exact McNemar two-sided $p = 0.25$. The paired bootstrap percentile 95% CI for the paired-success-rate difference (computed with `bootstrap_seed = 1729` and 10,000 resamples as recorded in analysis/analysis_log.jsonl) is [0.0000, 0.005838198498748957]. The CI lower bound reaching 0.0 reflects the high proportion of concordant successful pairs and the small number of discordant pairs; those data characteristics limit resolution for small absolute effects.

Missingness and failed calls: The preregistered missingness summary (analysis/missingness_summary.csv) reports `complete_pairs = 1,199`; `failed_baseline_episodes = 1`; `failed_guarded_episodes = 1`; `both_missing_pairs = 1`. The deterministic exclusion rules were not triggered for contamination; analysis/contamination_summary.csv is empty. The single paired exclusion implied by `both_missing_pairs = 1` is the accounting source for the one planned pair drop from 1,200 to 1,199 complete pairs.

Repair invocation diagnostics: Stage_counts (execution/manifest and deterministic_reconciliation.json) show `repair = 3` while `shared_initial_generation = 1,200`. The repair count matches repair_provider traces present for exactly three episodes in the execution/provider_calls and execution/scoring artifacts. Scorer and validator traces (examples in execution/scoring/*.json) report `validation_after.valid = true` for the majority of episodes and `repair_applied = false`, indicating that initial candidates typically satisfied deterministic_netops_validator_v5 checks and therefore required no automated repair.

Representative artifact evidence: Representative task manifest entries and scoring artifacts illustrate the evidence pattern. For example, execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json show identical `shared_initial_candidate` content and validator traces with `validation_after.valid = true` and `repair_applied = false`; difficulty for that task is labeled "medium" in the task_manifest entry. Examples with higher difficulty labels ("hard", "very_hard") appear in representative task entries (task-000002 level "hard",

task-000003 level "hard") and still produced accepted shared initial candidates in many cases. These artifact-level diagnostics demonstrate (a) heterogeneous declared task difficulty across the corpus, and (b) that under the chosen deterministic task generator and validator settings the initial LLM output frequently met intent constraints.

Interpretation of discordant pattern: The observed discordant pattern ($n_{10} = 3$, $n_{01} = 0$) means all discordant pairs favored the guarded pipeline; however, the absolute discordant count ($3/1,199 \approx 0.25\%$) is too small to yield statistical significance under the prespecified inferential thresholds. Under the exact McNemar framework the two-sided $p = 0.25$ (archived), reflecting the binomial probability of observing such an imbalanced discordant split given the small number of discordant pairs. The paired bootstrap CI's narrow upper bound (≈ 0.00584) but lower bound at zero quantifies that any true absolute benefit, if present, is small in magnitude within this corpus and validator configuration.

Reproducibility anchors: All numbers above match archived artifacts: analysis/paired_contingency_table.csv ($n_{11}/n_{10}/n_{01}/n_{00}$), analysis/condition_summary.csv (per-condition counts and success rates), analysis/results.json (estimate, p-value, CI, bootstrap parameters), execution/execution_manifest.json (provider audit counts and timestamps), and representative scoring artifacts (execution/scoring/*.json) that record validator trace fields `validation_before/validation_after`, `repair_applied` flags, and `normalized_configuration` values. The run's bootstrap parameters (`seed = 1729`, `resamples = 10,000`) and analysis log are included in analysis/analysis_log.jsonl to enable exact reproduction of the CI reported above.

V. DISCUSSION

Statistical and operational interpretation: The exact McNemar test and bootstrap CI both indicate no statistically significant advantage for the guarded pipeline under this corpus and validator configuration ($p = 0.25$; 95% CI includes zero). Two operational explanations are supported by archived artifacts. First, near-ceiling baseline performance ($\approx 99.75\%$) constrained absolute headroom for improvement; the pre-registered MDE was 5 percentage points, so the run was not powered to detect the observed ≈ 0.25 percentage-point effect. The archived CI and discordant counts ($n_{10} = 3$, $n_{01} = 0$) quantify this limitation and show that detection of much smaller practical gains would require substantially more discordant pairs or a task corpus with lower baseline success.

Second, corpus and verifier design materially affected repair invocation. Representative task_manifest entries and scoring traces show tasks were synthetic with explicit required sequences and conservative initial generators (deterministic_netops_task_generator_v5), and the deterministic validator (deterministic_netops_validator_v5) accepted the majority of initial candidates. Those design choices—synthetic tasks yielding clear required sequences and a validator configuration that enforces the preregistered intent constraints—likely reduced baseline difficulty and therefore the frequency of repairs.

The provider audit (repair = 3) and scoring artifacts (validation_after.valid = true for typical episodes) jointly support this interpretation.

Threats to validity and externality: Internal validity is strong for deterministic components (frozen seeds, deterministic task generation, and automated exclusion rules), but construct validity depends on whether the binary success rule (validator acceptance + emulator availability tolerance + no rollback) maps to operationally meaningful correctness across production environments. External validity is limited: only one requested model family (gpt-5-mini) was exercised, task examples were synthetic/consented and vendor-agnostic, and emulation (Mininet-style) may not capture vendor idiosyncrasies and routing convergence phenomena. Conclusion validity: the very low discordant counts imply limited power to detect small absolute differences; the preregistered MDE (5 pp) should be considered when interpreting the null result.

Operational implications: When initial generation already meets deterministic verifier criteria for common, well-specified tasks, generate-validate-repair will be infrequently exercised and measured gains on a binary success metric will be small. To evaluate verifier value, future benchmarks should increase task ambiguity/adversarial content, vary verifier strictness, and exercise multiple model families. The archived artifacts and traces provide a reproducible starting point to perform such sensitivity studies.

VI. LIMITATIONS

- Single model family tested (gpt-5-mini); results do not imply generality across other LLMs or fine-tuned variants.
- Task corpus skew: synthetic/consented tasks yielded near-ceiling baseline success ($\approx 99.75\%$), limiting headroom and statistical power to detect practical improvements by guarded pipelines.
- Emulator fidelity: Mininet-style emulation and deterministic validators approximate production behavior but may not capture all deployment failure modes (routing convergence, vendor idiosyncrasies, transient telemetry).
- Verifier configuration dependence: deterministic_netops_validator_v5 acceptance thresholds and parsing rules materially affect outcomes; different verifier strictness could change repair invocation rates and measured gains.
- Analytic deviation documented: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived confirmatory analysis used an exact McNemar test because outcomes were binary. This deviation is recorded in the archived analysis artifacts and in the Disclosure Statement above.
- Operational external validity: absence of heterogeneous vendor renderers, real production templates, and live rollouts constrain direct production generalization.

VII. CONCLUSION

We preregistered and executed NetOpsTraceBench, a paired, emulator-grounded study comparing one-shot and verifier-assisted NL $\rightarrow$ configuration pipelines on 1,199 completed pairs. Under our synthetic task corpus, deterministic validator settings, and gpt-5-mini as requested, baseline one-shot generation already achieved near-ceiling success ($\approx 99.75\%$). The guarded pipeline increased absolute success by ≈ 0.25 percentage points (paired difference = 0.0025020851) with exact McNemar two-sided $p = 0.25$ and a 95% bootstrap CI that includes zero. Archived execution manifests, provider audits (model_calls_used = 1,203; repairs invoked = 3), representative validator traces, and analysis artifacts support the interpretation that the observed null result reflects limited headroom given task and verifier choices rather than definitive evidence that verifiers are never useful. Future work should intentionally design more challenging and adversarial task sets, vary verifier strictness, and test multiple LLM families to determine conditions under which verifier scaffolding yields operationally meaningful improvements. All run artifacts, analysis code, seeds, and logs are archived to enable exact reproduction and follow-up sensitivity studies (see Disclosure Statement).

DISCLOSURE STATEMENT

Mandatory disclosure (preregistered run rX4f7-1). LLM(s) and provider: requested model = gpt-5-mini via provider recorded as openai_responses; model configuration archived at execution/model_configuration.json (requested_model=gpt-5-mini, max_output_tokens=1500). Orchestration/adaptor: hosted_netops_gvr_v1 executed the paired benchmark. Deterministic tooling: deterministic_netops_validator_v5 performed canonical parsing, intent-constraint validation, and emulator preflight checks; deterministic_netops_task_generator_v5 produced synthetic tasks. Exact initial master prompt: archived at provenance/master_prompt.txt (immutable reference SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd51582bb39b15ba). Topic constraint and autonomy boundary: the human Research Director selected the topic family and approved the master prompt pre-lock; after the autonomy lock the AI autonomously formulated the research question, conducted literature review, generated the preregistration, executed the experiment, performed analysis, and wrote and revised the manuscript. Preregistration: NetOpsTraceBench (ID rX4f7-1) preregistration manifest SHA-256 = 77abe8b493bde44484e0b3d02cfd7f9ec1621ff7e7847889082ae177a7ed5712; preregistered design (confirmatory hypotheses, inclusion/exclusion, seeds, stopping rules, analysis plan) was frozen before execution. Execution summary: started 2026-08-30T01:21:55.415955+00:00, completed 2026-08-30T03:15:12.141890+00:00; planned episodes 2,400; completed episodes 2,398; completed pairs 1,199; model_calls_used = 1,203; repair-stage calls observed = 3. Analysis artifacts and deviation record: the preregistration specified a Wilcoxon signed-rank paired procedure; the

archived executed confirmatory analysis used an exact McNemar two-sided test on paired binary outcomes plus a paired bootstrap percentile CI. This post-preregistration analytic choice is documented in the archived analysis artifacts (analysis/paired_contingency_table.csv, analysis/results.json, analysis/analysis_log.jsonl, analysis/paired_difference_figure.svg) produced as part of the run that completed at 2026-08-30T03:15:12.141890+00:00. Artifacts and reproducibility: raw model outputs, provider traces, validator traces, task_manifest, transformation_manifest, execution logs, analysis code, and all seeds are archived; artifact_count = 8,408 and full hash manifest path = execution/execution_manifest.json. Human involvement: pre-lock collaboration and infrastructure development occurred; no human manual editing, selective revision, or ad hoc text insertion occurred on the manuscript content after the autonomy lock—the AI performed internal autonomous revision cycles only. Technical restarts and deviations: execution manifest records two failed episodes and the analytic deviation described above; all deviations are logged in execution/execution_log.jsonl. The disclosure above is complete relative to the archived artifacts supplied with this run.

REFERENCES

- [1] <http://hdl.handle.net/20.500.12708/229494>
- [2] <https://arxiv.org/abs/2608.13389>
- [3] <http://arxiv.org/abs/2403.09369>
- [4] <https://doi.org/10.2139/ssrn.6947162>
- [5] <https://doi.org/10.2139/ssrn.7222278>
- [6] <https://doi.org/10.20935/acadai8260>
- [7] <https://doi.org/10.3390/info17030297>
- [8] <https://doi.org/10.3390/app16010085>